

黑白棋（Reversi）大作业报告

2300013176 钱宇鹏



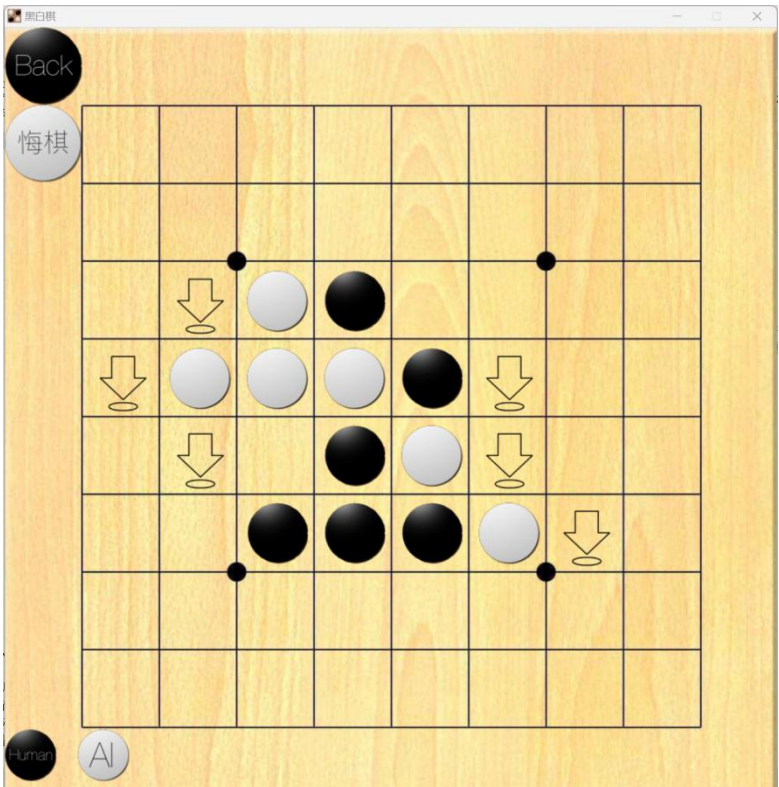
一、游戏介绍

如上图所示，在打开程序后的主界面有多个按钮。所有交互均通过点击鼠标左键进行。在右上方可以选择开始新游戏：与 AI 对战并选定执黑或执白，也可以进行玩家对战(PVP)，点击“Exit”按钮即退出程序；右下方的按钮进行存档读取，共设置了 3 个存档供读取 (用文件实现)；左边的“Settings”按钮可以进入设置界面，可以设置 AI 难度和 bgm。



(图为设置界面)

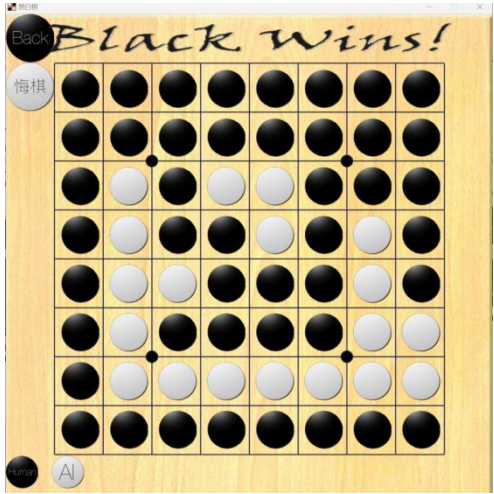
点击右上方的按钮即可选择与 AI 对战/玩家对战并开始新游戏。进入对局后由执黑方开始轮流落子。



(图为游戏界面)

游戏界面如图所示。每次落子时，会有箭头提示可以落子的地方（图中此时提示执黑方落子）；鼠标左键点击对应棋盘格即落子。左下角的黑白两颗棋子分别显示执黑方与执白方为 AI 还是人类；如上图即为玩家执黑，AI 执白。（注：困难 AI 对战时可以在终端显示 AI 此次落子的实时胜率）

点击棋盘左上方的“Back”按钮会提示存档并退回主界面；“悔棋”按钮可将棋盘状态退回上一回合（即撤回对方的一步落子和自己的一步落子），每次对局限制只可悔棋 3 次。存档后在主界面读档可以回到存档时棋盘状态继续进行游戏；游戏结束后会显示胜者，暂停 5 秒后回到主界面。



(图为存档界面和游戏结束画面)

二、代码逻辑

本次大作业使用 C++ 中的图形库 easyx 进行 GUI 的编写。程序编写方面，使用了 Dev-C++ 中的项目，源代码部分共分为 7 个文件：1 个 .cpp 文件——main.cpp，用以进行主函数的调用；6 个 .h 文件——initialize.h、graph.h、game.h、AI_ez.h、AI_medium.h、AI_hard.h，分别负责初始化、图形界面生成和交互、游戏规则逻辑、简单 AI、中等 AI、困难 AI 部分的函数。下面分别就这 7 个文件介绍其中的函数与编写逻辑。

1) main.cpp (主函数)

```
int main();  
//打开程序并调用初始化主界面函数
```

2) initialize.h (初始化)

```
struct node{};  
//为 MCTS 算法准备的节点记录, 记录有此时对局状态和其父、子节点位置  
struct file{};  
//用于存档的结构体  
void initialize();  
//初始化函数, 导入图片、设置参数  
void fileout();  
//写入存档文件  
void filein();  
//读取存档文件
```

```
struct node {  
    int chess[10][10];  
    int color; //此时轮到color走子  
    int x, y; // (x,y) 为该节点落子位置  
    int qual, simu; //qual为该节点胜利次数,simu为该节点模拟次数  
    vector<int> son; //子节点位置  
    int father; //父节点位置  
}v[200001];  
  
struct file{  
    int chess[10][10][128]; //file.chess[i][j][k]表示在x步后棋盘(i,j)处的状态(跳步算一步)  
    int black,white; //记录对战者,0表示玩家,1为ai_ez,2为ai_medium,3为ai_hard  
    int step; //表明对局进行了几步  
}save[4];  
//save[0]是cache,save[1/2/3]是存档1/2/3
```

(图为结构体初始化定义)

3) graph.h (图形界面)

```
void drawAlpha(IMAGE *picture, int picture_x, int picture_y);  
//抠除透明图形的边框  
void update();  
//更新棋盘  
void cover();
```

```

//封面界面
void savecover();
//询问存档界面
void saveread(int num);
//读取 num 号存档
void setting();
//设置界面

```

4) game.h (游戏规则与逻辑)

```

void place();
//玩家落子
void black();
//分配执黑落子权, 选择调用 place() 还是 aiplace_()
void white();
//分配执白落子权, 同上
void first();
//开始新游戏之前对棋盘的初始化
void start();
//开始游戏, 在!end()的情况下轮流调用 black() 和 white() 以实现轮流落子
bool search(int x, int y, int d, int color);
//找寻 d 方向是否能“夹”, 通过递归进行判断
bool acc(int x, int y, int color);
//判断 (x, y) 处 color 能否落子, 通过调用 8 个方向的 search(x, y, d, color)
void turn(int x, int y, int color);
//落子后翻转中间“夹”的棋子
bool check(int color);
//判断 color 是否有地方落子, 通过调用棋盘上各个位置的 acc(i, j, color)
bool end();
//判断游戏是否结束, 通过调用 check(1) 与 check(-1) 进行判断

```

5) AI_ez.h (贪心 AI)

```

int turnamount(int x, int y, int color);
//计算 color 在 (x, y) 处落子后的总收益
void aiplace_ez();
//ez 版本 AI 落子(贪心), 找出最大的 turnamonut(i, j, color)+value[][] 和

```

6) AI_medium.h (min-max 搜索 AI)

```

int step[10][10];
//定义 step[][] 数组用于存储 AI 模拟计算时的“棋盘”状态

```

¹ 1 表示黑色, -1 表示白色

```

bool stepsearch(int x, int y, int d, int color);
//对 step[i][j]的是否能“夹”的判断, 同 search(x, y, d, color)
void stepturn(int x, int y, int color);
//对 step[i][j]的翻转, 同 turn(x, y, color)
bool stepacc(int x, int y, int color);
//同 acc(x, y, color)
bool stepcheck(int color);
//同 check(color)
bool stepend();
//同 end()
int stepamount(int x, int y, int color);
//同 turnamount(x, y, color)
int evaluate(int color);
//局面价值评估函数
void aiplace_medium();
//medium 版本 AI 落子(min-max 搜索), 通过搜索 6 步之后使得总局面价值
函数值最大的选择进行落子, 通过调用逐层调用 aisearch() 和
playersearch() 实现
int aisearch(int color, int depth, int alpha, int beta);
//max 层搜索(color 走子颜色, depth 搜索深度, alpha&beta 用于剪枝), 返回
值是 该层最大价值, 调用 evaluate(color) 进行评估
int playersearch(int color, int depth, int alpha, int beta);
//min 层搜索, 返回值是 该层对 AI 最小价值(即对玩家最大价值), 其余同上

```

7) AI_hard.h (蒙特卡洛树搜索 AI)

```

void MCTS() { //执行一次MCTS过程
    int placement = 0;
    bool simresult;

    while (!expandable(v[placement]) && v[placement].son.size() > 0) {
        //cout<<v[placement].son.size()<<"sonsize\n";
        placement = selection(placement);
        //cout<<placement<<"selection\n";
    }
    //selection到一个未访问节点

    if (!MCTSend(v[placement]) && expandable(v[placement])) { //将该节点expansion(如果游戏在该节点还未结束)
        expansion(placement); //simulation并且backtrace
        simresult = simulation(v[nodecnt]);
        backtrace(nodecnt, simresult);
    } else {
        simresult = simulation(v[placement]);
        backtrace(placement, simresult);
    }

    return;
}

```

(图为 MCTS 过程部分源代码)

```

bool MCTSsearch(node V, int x, int y, int d);
//对节点 V 是否能“夹”的判断, 同 search(x, y, d, color)
bool MCTSacc(node V, int x, int y);
//同 acc(x, y, color)

```

```

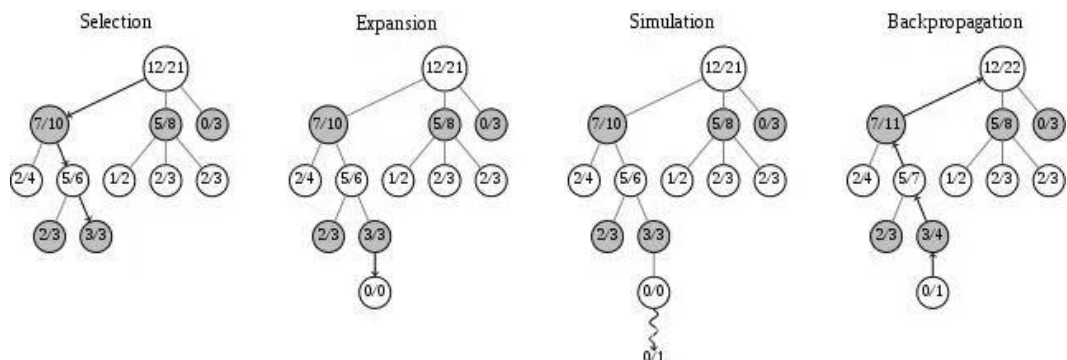
bool MCTScheck(node V);
//同 check(color)
bool MCTSend(node V);
//同 end()
void aiplace_hard();
//hard 版本 AI 落子(MCTS), 对当前局面进行 times=12000 次 MCTS 过程(即调用 MCTS())
void MCTS();
//执行一次 MCTS 过程, 每次通过调用 selection(place) 选择到一个可扩展
//的节点, 使用 expansion(place) 对该节点进行扩展并使用 simulation(V) 模
//拟局面判断 AI 胜负, 之后通过 backtrace(place, win) 更新沿途各个节点的
//模拟次数和胜利次数统计
void expansion(int place);
//扩展一个叶子节点
bool simulation(node V);
//对一个节点 V 进行模拟, 随机进行一局棋局, 记录胜者
int selection(int place);
//根据 UCT 函数选择一个子节点
void backtrace(int place, bool win);
//回溯, 向上更新回溯结果(每个节点的 simu&qual)直到根节点
float UCT(int place);
//UCT 函数, 判断每个节点成为最优节点的期望值(影响因素: 节点胜率, 鼓励
//探索项, 位置优势奖励项)
bool expandable(node V);
//判断节点 V 是否可扩展

```

三、算法解释

主要进行蒙特卡洛树搜索部分算法的讲解。

MCTS 算法的设计思路是：每一步进行落子时，筛选出几种“最可能”的走法，再筛选这些走法之后对手“最可能”的走法，如此重复进行。MCTS 算法经历如下四个过程：



(图为 MCTS 过程示意图)

1) 选择 (Selection):

从根结点 R 开始, 选择连续的子结点向下至叶子结点 L。下面的结点有更多选择子结点的方法, 使游戏树向最优点扩展移动, 这是蒙特卡洛树搜索的本质。

2) 扩展 (expansion):

除非任意一方的输赢导致游戏结束, 否则 L 会创建一个或多个子结点或从结点 C 中选择。

3) 模拟/仿真 (Simulation):

在结点 C 中进行随机布局。

4) 回溯/反向传播 (Backpropagation):

使用布局结果更新从 C 到 R 的路径上的结点信息。

在选择子节点时使用了 UCT 算法:

$$UCT(v_i, v) = \frac{Q(v_i)}{N(v_i)} + c \sqrt{\frac{\log(N(v))}{N(v_i)}} \quad (\text{图为 UCT 函数表达式})$$

在上图中, $Q(v)$ 是节点 v 赢的次数, $N(v)$ 是节点 v 模拟的次数, c 是一个常数(通常在 1 到 2 之间)。

该函数描述了各个节点在选择时的优劣: 第一项为该节点胜率, 这是 MCTS 算法的核心: 选择胜率最高的节点; 但仅根据先前模拟得到的胜率选择之后的模拟策略会导致一些实际胜率较低的节点在先前随机模拟时碰巧获得了高胜率, 从而使 MCTS 搜索过程走向“歪路”。

为避免这种情况出现, 添加 UCT 函数的第二项: 此项用于鼓励探索, 使得扩展次数很少的节点获得更高的 UCT 函数值, 以使得 MCTS 过程的模拟更加稳定和准确。 c 为常数, 用于调控对节点探索的鼓励程度。 c 值过低会使得鼓励探索效果不佳, c 值过高又会使得高胜率节点的优势难以显现: 因此参数 c 需要平衡调控, 本次设置为 $c=1.48$ 。

同时, 本次为了落实黑白棋的位置优先策略, 额外添加 UCT 函数的第三项:

$$-c1 \cdot placevalue(x, y) \cdot color$$

此项用于奖励位置优势, 其中 $placevalue(x, y)$ 表示为在 (x, y) 处落子的优先级数值, $color$ 表示落子颜色与 AI 执棋颜色是否相同。这样, 可以使得 MCTS 过程和最终落子朝位置更优的方向倾斜, 以更加直接的方式落实了位置优先策略。

下图为落子位置优先级表:

1	5	3	3	3	3	5	1
5	5	4	4	4	4	5	5
3	4	2	2	2	2	4	3
3	4	2			2	4	3
3	4	2			2	4	3
3	4	2	2	2	2	4	3
5	5	4	4	4	4	5	5
1	5	3	3	3	3	5	1

四、参考文章/网页

(1)

【深度解析黑白棋 AI 代码原理(蒙特卡洛搜索树 MCTS+Roxanne 策略) - CSDN】

<http://t.csdnimg.cn/mPlaU>

(2)

EasyX 使用方法及函数&功能:

<https://docs.easyx.cn/zh-cn/style-settings>